



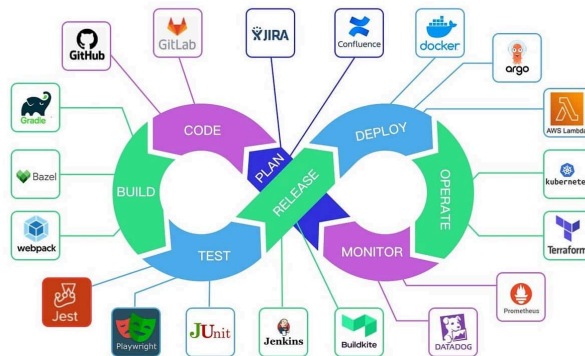
CI Fundamentals and Code Quality

CI/CD WorkShop

今天你會學到什麼

1. 什麼是 CI / CD
 - 為什麼需要自動化流程
2. **Continuous Integration 流程**
 - CI 與開發日常的關係
3. **軟體開發週期簡介**
 - 從需求 → 開發 → 測試 → 部署
4. **在 GCP 部署 SonarQube (Community Edition)**
 - Docker + GCE 建置流程
 - Nginx 安裝與設定
5. **SonarQube 設定與使用**
 - 專案設定
6. **設計團隊的測試流程與 GitHub Actions**
 - CI pipeline 架構設計
 - Java Spring Boot API 範例
7. **GitHub Actions 設定實作**
 - yml 設定檔拆解與範例
8. **SonarQube 報告解讀**
 - 代碼品質、測試覆蓋率、異常檢測

什麼是 CI / CD



CI (持續整合 Continuous Integration)

自動化程式碼整合與驗證流程

- 程式建置：每次 Commit 自動編譯，確保環境一致性
- 品質檢查：SonarQube 程式碼品質分析
- 安全掃描：自動檢測程式碼漏洞
- 自動測試：單元測試 + 整合測試驗證

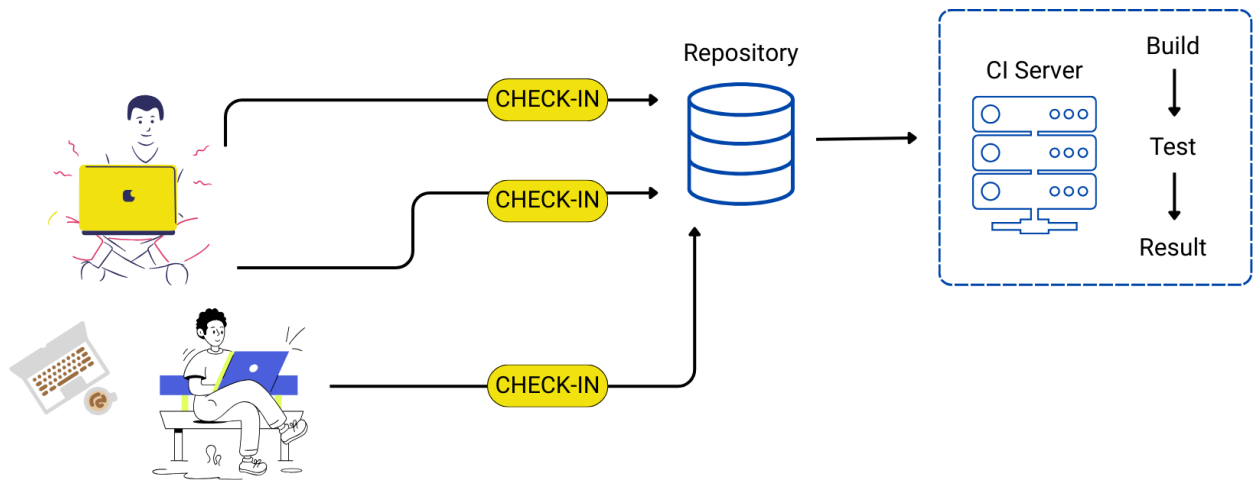
CD (持續部署 Continuous Deployment)

自動化應用程式部署與監控

- 容器化打包：Docker 統一部署格式
- GitOps 管理：聲明式部署配置
- 多環境晉升：Dev → Staging → Production
- 監控告警：自動化健康檢查

無論是 CI or CD 就是在開發與部署的流程自動化。因為專案的進版、測試（以前是人為手動，進而寫程式為自動化單元測試）、部署、觀察狀況，在沒有自動化以前，人為做這些事情，耗時、容易出錯；現在的社會環境的軟體開發工程也與幾十年前不同（以前較多 AP Standalone，現在很多分散式系統）所以需要自動化程序來幫助工程師，開發多一點功能❌ 多些時間打電動✅

Continuous Integration 流程



軟體開發生命週期 (SDLC) 簡介



Step 1: 規劃 (Planning)

- 定義需求與目標
- 收集利害關係人需求
- 確定開發範圍與優先級

Step 2: 設計 (Design)

- 系統架構設計
- 技術方案規劃

- 確保可擴展性與效能

Step 3: 開發 (Development)

- 程式碼實作
- 功能開發與整合
- 遵循編碼標準

Step 4: 整合與測試 (Integration & Testing)

- 單元測試、整合測試
- 開發者主導測試
- 自動化測試與持續驗證

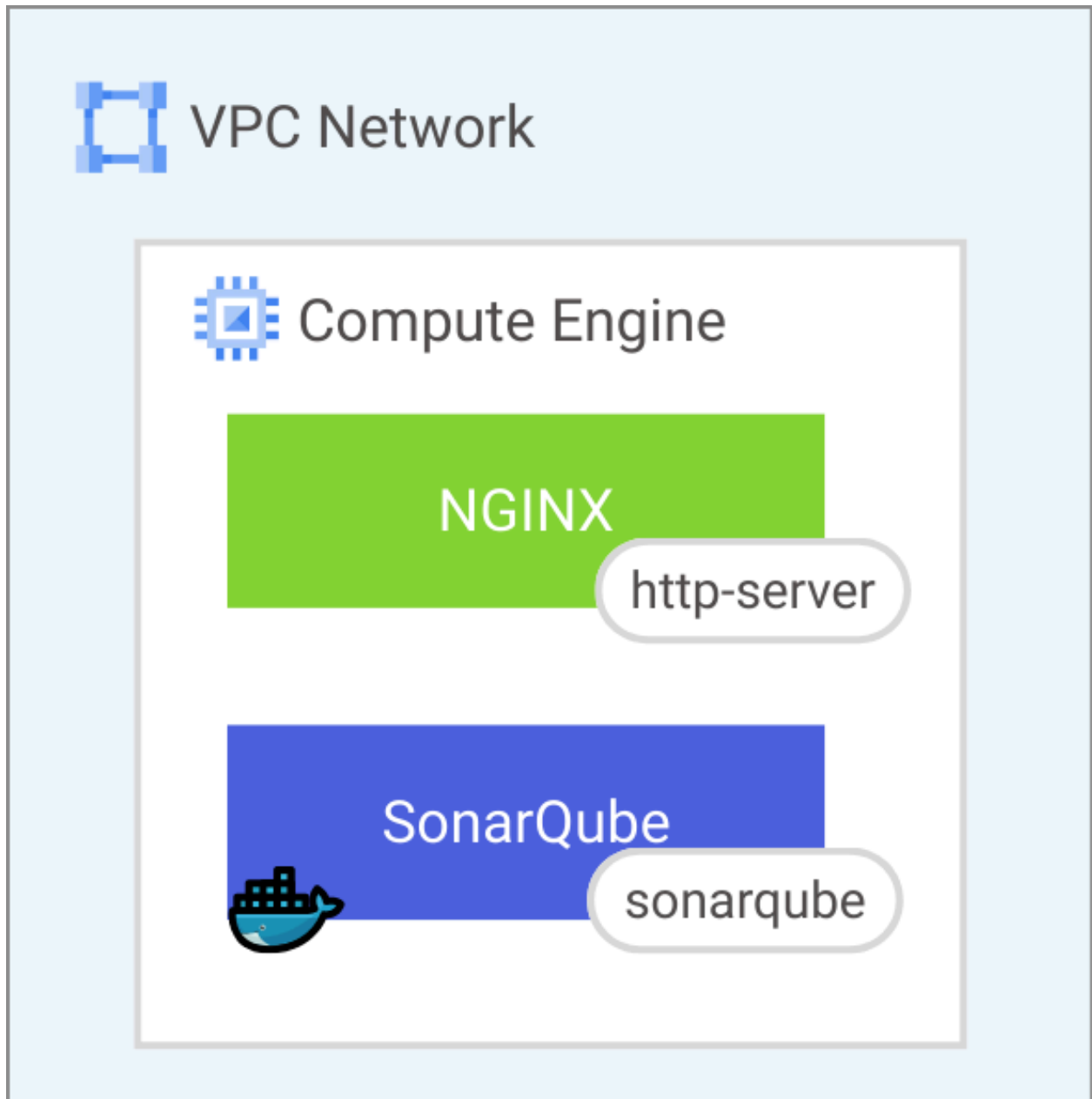
Step 5: 部署與維護 (Deployment & Maintenance)

- 生產環境部署
- 監控與維護
- 持續改進與優化

現代 SDLC 特點

- 自動化測試與持續驗證
- 自動化建置與部署
- 全流程自動化
- 快速迭代與回饋（敏捷開發）

GCP 上部署 SonarQube (Community Edition)



建立 VM

- 建立 **Compute Engine VM** (建議 2 vCPU / 8GB RAM)
- OS: Ubuntu 22.04 LTS
- 設定 **static IP**

- 加上 **network tag** `sonarqube` 、 `http-server`

設定防火牆

到 **VPC Network → Firewall rules → Create Rule**

- Name: `allow-sonarqube`
- Targets: `Specified target tags` → `sonarqube`
- Source filter: 你的 IP
- Protocols & ports: `tcp:9000`

安裝 Docker 與 Docker Compose

VM 啟動後，SSH 進去：

```
# 更新
sudo apt update && sudo apt upgrade -y

# 安裝 Docker
sudo apt install -y docker.io docker-compose

# 啟動與自動開機
sudo systemctl enable docker
sudo systemctl start docker
```

建立 docker-compose.yml

在 VM 上建立目錄：

```
mkdir -p ~/sonarqube
cd ~/sonarqube
```

建立 `docker-compose.yml`：

```
version: "3"

services:
  sonarqube:
```

```
image: sonarqube:25.8.0.112029-community
container_name: sonarqube
ports:
  - "9000:9000"
depends_on:
  - db
environment:
  SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
  SONAR_JDBC_USERNAME: sonar
  SONAR_JDBC_PASSWORD: sonar
volumes:
  - sonarqube_data:/opt/sonarqube/data
  - sonarqube_extensions:/opt/sonarqube/extensions
  - sonarqube_logs:/opt/sonarqube/logs
restart: always

db:
image: postgres:15
container_name: sonarqube_db
environment:
  POSTGRES_USER: sonar
  POSTGRES_PASSWORD: sonar
  POSTGRES_DB: sonar
volumes:
  - postgresql:/var/lib/postgresql
  - postgresql_data:/var/lib/postgresql/data
restart: always

volumes:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:
```

啟動 SonarQube

```
docker-compose up -d
```

檢查容器狀態：

```
docker ps
```

SonarQube 預設在 `http://<Static-IP>:9000`
(帳號密碼：`admin / admin`)

SonarQube 本機綁定 bind mount 的安裝方式

改成 本機目錄綁定 (bind mount)，這樣所有資料會存在 `/sonarqube` 目錄底下，方便你備份或清理。

假設你的 `docker-compose.yml` 在 `/sonarqube` 目錄底下，建議先建立幾個子資料夾：

```
cd /sonarqube
mkdir -p sonarqube_data sonarqube_extensions sonarqube_logs
mkdir -p postgresql postgresql_data

echo "設定資料夾權限..."
chmod -R 777 sonarqube_data sonarqube_extensions sonarqube_logs
chmod -R 777 postgresql postgresql_data
```

`docker-compose.yml` 改成：

```
version: "3"

services:
  sonarqube:
    image: sonarqube:25.8.0.112029-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    depends_on:
      - db
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    volumes:
      - ./sonarqube_data:/opt/sonarqube/data
      - ./sonarqube_extensions:/opt/sonarqube/extensions
      - ./sonarqube_logs:/opt/sonarqube/logs
    restart: always

  db:
    image: postgres:15
    container_name: sonarqube_db
    environment:
      POSTGRES_USER: sonar
```

```
POSTGRES_PASSWORD: sonar
POSTGRES_DB: sonar
volumes:
  - ./postgresql:/var/lib/postgresql
  - ./postgresql_data:/var/lib/postgresql/data
restart: always
```

說明

- `./sonarqube_data` → 存放 SonarQube 的索引資料
- `./sonarqube_extensions` → 存放插件 (你安裝的額外語言包都會在這裡)
- `./sonarqube_logs` → 存放 SonarQube 日誌
- `./postgresql_data` → **最重要**，存放 PostgreSQL 的真實 DB 資料
- `./postgresql` → 存放 PostgreSQL 其他運行需要的檔案

若你已經用 **舊版 (Docker Volume 方式)** 跑起來了

現在想改成 **新版 (Bind Mount，直接掛本機目錄)**

這種情況要做的，就是把舊容器停掉 → 移除 → 再用新版設定啟動。步驟如下：

1. 停止舊容器

```
docker-compose down
```

這會停掉 `sonarqube` 和 `sonarqube_db`，但資料還在 Docker volume 裡。

2. 清掉舊的 volume

如果你想要「重新開始」(資料全部重來，Postgres 資料庫也清空)：

```
docker volume rm sonarqube_sonarqube_data sonarqube_sonarqube_extensions
```

3. 換成新版 `docker-compose.yml`

4. 啟動新容器

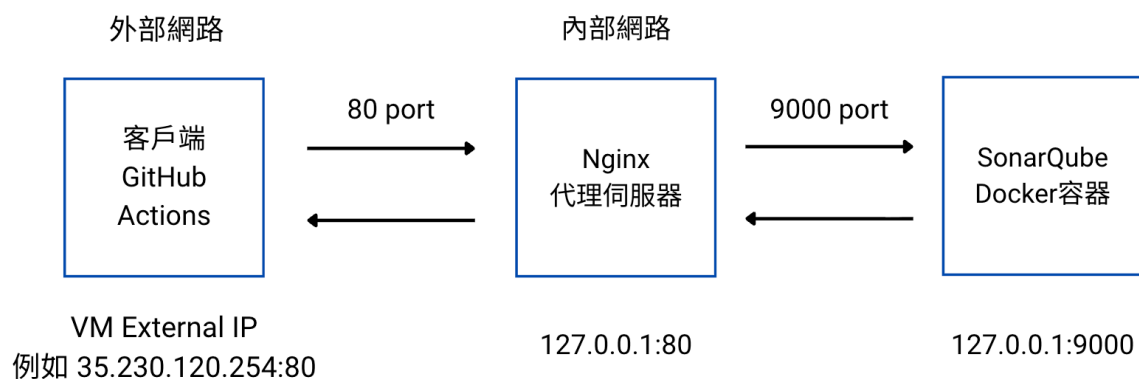
```
docker-compose up -d --build
```


Nginx 安裝與設定

這個 Nginx 設定的核心作用是：

- 只需開放 80/443 埠，也可以加入 SSL/HTTPS
- 轉發到內部 9000 port SonarQube
- SonarQube 不直接暴露在外網
- 可以加入認證、限制等安全措施
- 可以負載平衡多個後端 (參考 Nginx Load balance)

SonarQube 部署架構圖 - Nginx 反向代理配置



首先 VM 設定防火牆

到 VPC Network → Firewall rules → Create Rule

- Name: `allow-http`
- 方向: Ingress
- Targets: `Specified target tags` → `http-server`
- 來源 IP 範圍: `0.0.0.0/0`
- Protocols & ports: `tcp:80`

並且你的 VM **network tag** 加上標籤: `http-server`

在你的 GCE SSH 中執行：

```
# 1. 安裝 Nginx 和密碼工具
sudo apt update
sudo apt install nginx apache2-utils -y

# 2. 建立認證用戶（替換 sonaruser 為你的用戶名）
sudo htpasswd -c /etc/nginx/.htpasswd sonaruser
# 會要求你輸入密碼，記住這個密碼

# 3. 建立 Nginx 設定
sudo tee /etc/nginx/sites-available/sonarqube << 'EOF'
server {
    listen 80;
    server_name _;

    # 不使用基本認證，改用 IP 限制
    location / {
        proxy_pass http://127.0.0.1:9000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

# 4. 啟用設定
sudo ln -s /etc/nginx/sites-available/sonarqube \
    /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl enable nginx

# 5. 測試內部連線
curl -I http://localhost
```

詳細說明：

- **proxy_set_header Host \$host**
 - 保留原始的 Host 標頭
 - SonarQube 能知道客戶端請求的域名
- **proxy_set_header X-Real-IP \$remote_addr**
 - 記錄真實的客戶端 IP
 - `$remote_addr` = 發送請求的真實 IP
- **proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for**
 - 記錄請求經過的所有代理伺服器
 - 追蹤完整的請求路徑
- **proxy_set_header X-Forwarded-Proto \$scheme**
 - 記錄原始請求的協定 (http/https)
 - 讓後端知道前端使用的協定
- **proxy_connect_timeout 60s;**
 - 連接後端超時
 - 與 SonarQube 建立連接的最長時間
- **proxy_send_timeout 60s;**
 - 發送請求超時
 - 發送請求給 SonarQube 的最長時間
- **proxy_read_timeout 60s;**
 - 讀取回應超時
 - 等待 SonarQube 回應的最長時間

完成設定後，這些請求都會轉發到 SonarQube：

```
# 這些請求都會被轉發到 http://127.0.0.1:9000
curl http://35.230.120.254/
curl http://35.230.120.254/projects
curl http://35.230.120.254/api/system/status
```

完整的請求流程

1. 客戶端發送請求

客戶端 → http://35.230.120.254/projects

2. Nginx 接收並處理

Nginx (80埠) 接收請求
↓
檢查 server_name (接受所有域名)
↓
匹配 location / (符合 /projects)
↓
準備轉發請求

3. 設定標頭並轉發

原始請求: GET /projects HTTP/1.1
Host: 35.230.120.254

轉發請求: GET /projects HTTP/1.1
Host: 35.230.120.254 (保留原始 Host)
X-Real-IP: 1.2.3.4 (客戶端真實 IP)
X-Forwarded-For: 1.2.3.4 (代理鏈)
X-Forwarded-Proto: http (原始協定)

4. SonarQube 處理並回應

SonarQube (9000埠) 處理請求
↓
回傳結果給 Nginx
↓

SonarQube 設定

登入

第一次登入，預設帳號密碼：admin / admin

要你改成新密碼



Log in to SonarQube

Login *

Password *

[Go back](#)

[Log in](#)

Mode: Standard Experience

Mode

There are two options to reflect the health of all the projects in this instance: [Multi-Quality Rule \(MQR\) Mode](#) and [Standard Experience](#)

Changing the mode will change how issues are categorized and ranked based on the results of the analysis.

Standard Experience

Offers the traditional use of the rule types: bugs, code smells, and vulnerabilities, categorized with a single type and severity for each rule.

This approach focuses on assigning severity to a rule and its issues based on the single software quality (for example, security, reliability or maintainability) it has the largest impact on. This is the rule categorization used in SonarQube Community Build 9.9 LTA and earlier.

Multi-Quality Rule (MQR) Mode

Aims to more accurately represent the impact software has on all software qualities. Very few issues impact only a single software quality. For instance, most vulnerabilities are also bugs. And vice versa. The MQR mode maps each rule to each of the qualities it impacts, with a separate severity rating for each quality.

This approach focuses on ensuring the impact of an issue on all software qualities is clear, not just the most severe one.

Key: sonar.multi-quality-mode.enabled

Set up project for Clean as You Code

我們先選第一個 Use the global setting

Set up project for Clean as You Code

The new code definition sets which part of your code will be considered new code. This helps you focus attention on the most recent changes to your project, enabling you to follow the Clean as You Code methodology. Learn more: [Defining New Code](#) [🔗](#)

Choose the baseline for new code for this project

☒ Use the global setting

Previous version

Any code that has changed since the previous version is considered new code.
Recommended for projects following regular versions or releases.

☐ Define a specific setting for this project

☐ Previous version

Any code that has changed since the previous version is considered new code.
Recommended for projects following regular versions or releases.

☐ Number of days

Any code that has changed in the last x days is considered new code. If no action is taken on a new issue after x days, this issue will become part of the overall code.
Recommended for projects following continuous delivery.

☐ Reference branch

Choose a branch as the baseline for the new code.
Recommended for projects using feature branches.

[Back](#)

[Create project](#)

這個設定決定「什麼程式碼算是新程式碼」。

SonarQube 給你兩種模式：

- **Previous version**

- 當你標記一個 **版本/release** 後，SonarQube 會記錄。
- 下一次掃描時，任何跟前一版本相比有修改的程式碼都算「新程式碼」。

- 適合有明確版本週期（例如 1.0 → 1.1 → 1.2）的專案。
- **Number of days**
 - 用時間範圍判斷，例如設定「30 天」。
 - 在過去 30 天內修改的程式碼都算「新程式碼」。

Project → Create Local Project

1 of 2

Create a local project

Project display name * ⓘ



Project key * ⓘ

Main branch name *

The name of your project's default branch [Learn More](#) ⓘ

Cancel

Next

要和你的 Java 專案的 pom.xml 設定檔一樣的值

```
<sonar.projectKey>com.example:springboot-mysql-login-ci-sonar</sonar.projectKey>

<sonar.projectName>SpringBoot MySQL Login CI SonarQube</sonar.projectName>
```

產生 SONAR_TOKEN

SonarQube 右上角頭像 → My Account

Security

If you want to enforce security by not providing credentials of a real SonarQube user, you can generate a token. This will increase the security of your installation by not letting your analysis user's password be exposed.

Generate Tokens

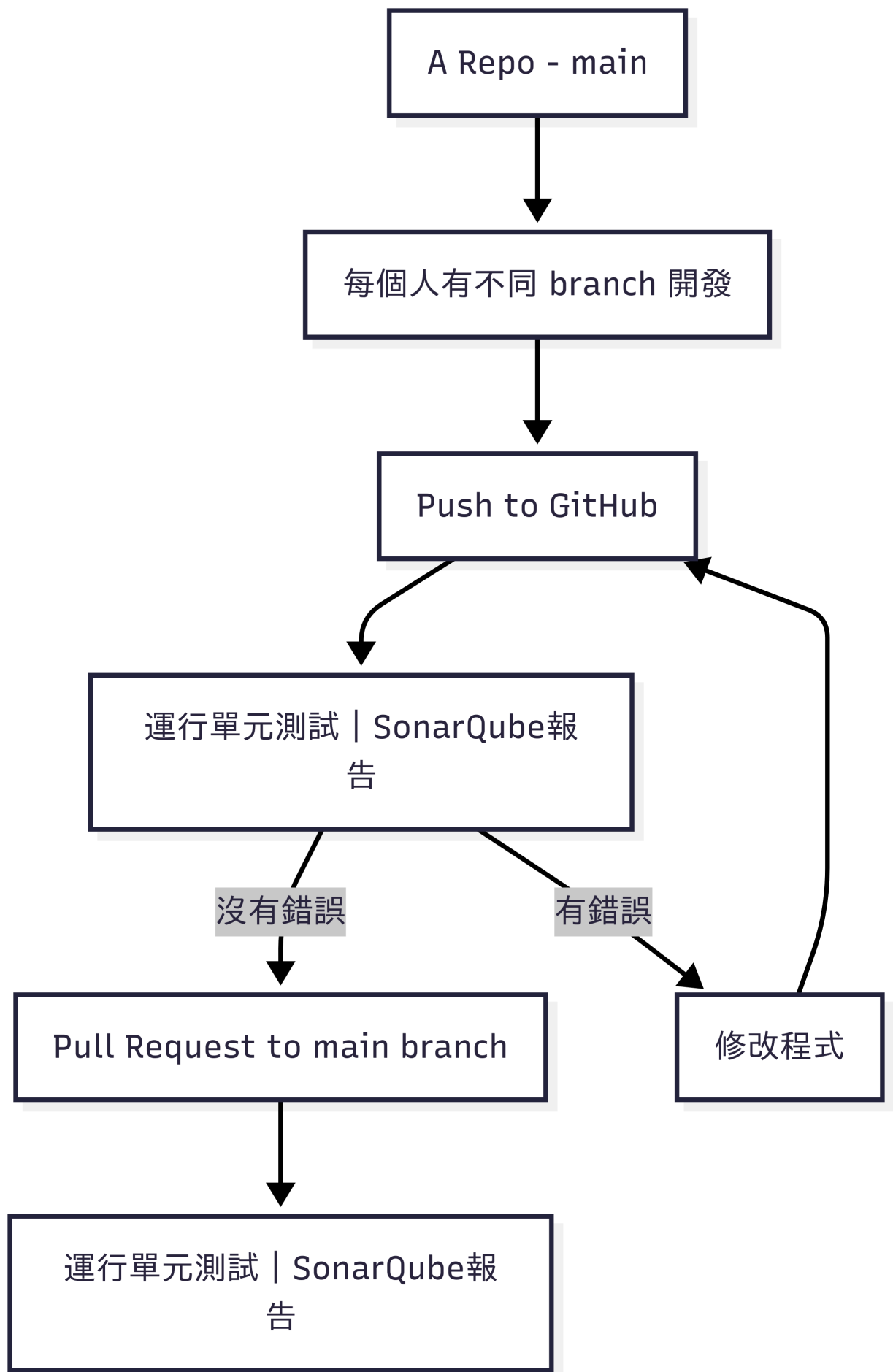
Name	Type	Expires in	
github-actions	User Token	90 days	<button>Generate</button>

產生 token → 複製貼到 GitHub Repository 專案的 **Settings** → **Secrets and variables** → **Secrets: SONAR_TOKEN**

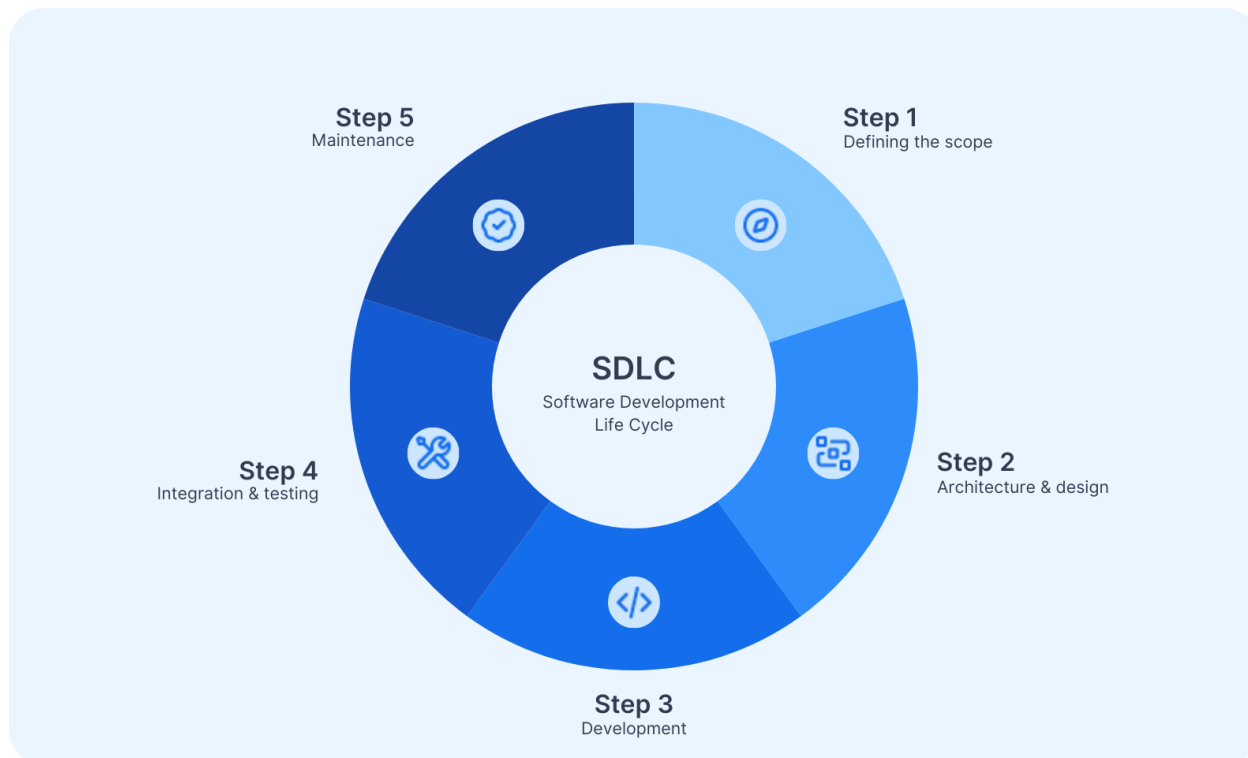
設計團隊的測試流程與 GitHub Actions 設定

每個團隊都有自己的開發與產品部署的流程，而這個流程會因為團隊的業務需求有所不同，沒有一定的標準答案，所以開始建置 CI/CD 的自動化流程，需要把團隊內部開發與部署情境疏離清楚。像是衛福部就是一個很好的例子。

我們先用一個簡單的例子



今天上課的內容就是在 軟體開發週期 SDLC 的 **Step 4. Integration & Testing**



專案 JAVA Spring Boot API 為例

這次上課的 [GitHub Repository](#)

重要的素材

- GitHub Actions 流程：`ci.yml`
 - `.github/workflow/ci.yml`
- Maven 設定檔：`pom.xml`
 - `sonar.projectKey:`
 - `sonar.projectName:`
- 程式碼單元測試

測試執行流程

CI Runner 啟動



mvn clean verify



編譯源代碼



運行單元測試



使用 H2 記憶體資料庫



生成 JaCoCo 覆蓋率報告



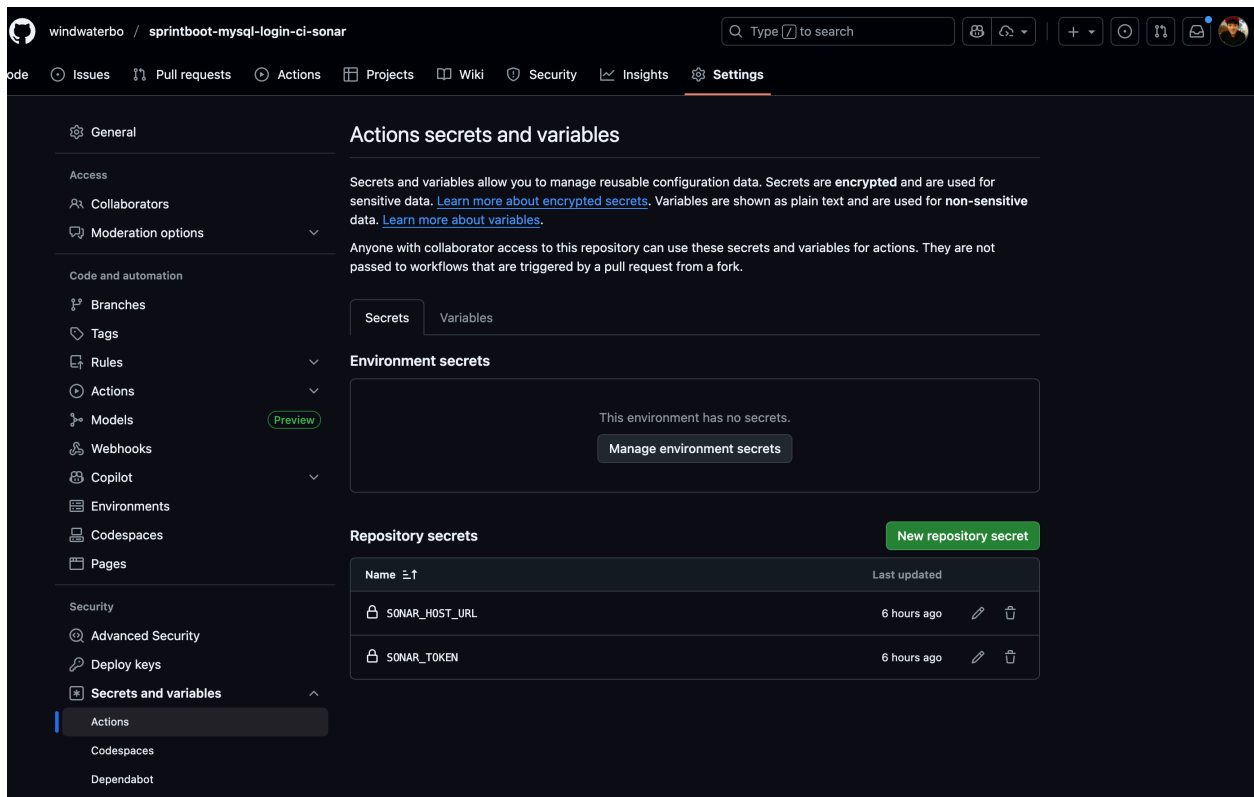
SonarQube 掃描



完成 CI 流程

GitHub Actions 設定

Repository 的 Settings



Settings → Secrets and variables

- SONAR_HOST_URL: 你的 SonarQube IP
 - http://35.230.120.254
- SONAR_TOKEN: SonarQube 產生的 user token
 - SonarQube 右上角頭像 → My Account
 - Security 頁籤
 - Generate Token :
 - Name: github-actions
 - Type: User Token
 - Expires: No expiration (或設定較長時間)

mvn clean verify 會執行：

- clean：清理之前的構建
- compile：編譯源代碼
- test：運行單元測試
- package：打包應用程式
- verify：運行整合測試（如果有的話）

單元測試使用 H2 記憶體資料庫

- 測試配置在 [application-test.properties](#) 中：

properties

spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driver-class-name=org.h2.Driver

- 測試使用 @ActiveProfiles("test")
- 測試環境自動使用 H2 記憶體資料庫
- 不需要外部 MySQL 連接

測試執行流程

graph TD

A[CI Runner 啟動] → B[mvn clean verify]

B → C[編譯源代碼]

C → D[運行單元測試]

D → E[使用 H2 記憶體資料庫]

E → F[生成 JaCoCo 覆蓋率報告]

F → G[SonarQube 掃描]

G → H[完成 CI 流程]

測試覆蓋率

CI 會生成 JaCoCo 覆蓋率報告：

- 位置：target/site/jacoco/jacoco.xml
- 用於 SonarQube 分析
- 顯示代碼覆蓋率統計

補充：

未來整合測試有需要 mysql, ci job 裡面就需要啟動 mysql

```
name: Java CI with Maven + SonarQube

on:
  pull_request:
    branches:
      - main
    types: [opened, synchronize, reopened]

jobs:
  build:
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: demo
        ports:
          - 3306:3306
        options: >-
          --health-cmd="mysqladmin ping -h 127.0.0.1 -proot"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=3

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup JDK
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17

      - name: Cache Maven packages
```


SonarQube 報告簡介

Type 類型說明

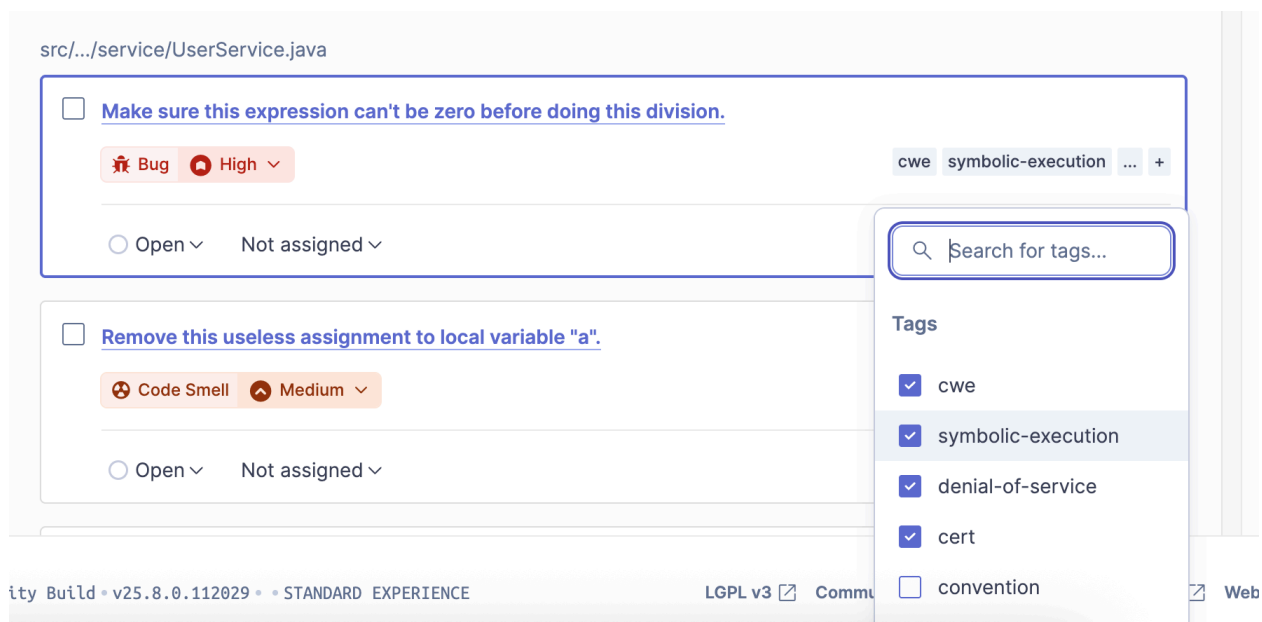
Bug (錯誤)

- 定義: 會導致程式執行時出錯或產生意外行為的程式碼問題
- 影響: 功能性問題，可能導致應用程式崩潰或產生錯誤結果
- 範例:

```
// Null Pointer Exception 風險
String name = getName();
return name.toLowerCase(); // name 可能為 null

// 資源未關閉
FileInputStream fis = new FileInputStream("file.txt");
// 忘記關閉 fis.close()
```

SonarQube 中這些標籤的含義，這些都是重要的安全和程式碼品質分類標準



SonarQube 標籤解釋

CWE (Common Weakness Enumeration)

- **定義:** 通用弱點枚舉，是一個標準化的軟體安全弱點分類系統
- **用途:** 用於識別、分類和追蹤常見的安全漏洞
- **範例:**
 - CWE-20: 輸入驗證不當
 - CWE-79: 跨站腳本攻擊 (XSS)
 - CWE-89: SQL 注入
- **意義:** 當看到這個標籤，表示此問題與已知的安全弱點模式相關

Symbolic-Execution

- **定義:** 符號執行，是一種程式分析技術
- **工作原理:** 使用符號值而非具體值來執行程式，找出潛在的執行路徑和問題
- **檢測內容:**
 - 除零錯誤 (如您當前的問題)
 - 空指標引用
 - 陣列越界
 - 死代碼路徑
- **意義:** SonarQube 使用此技術來深度分析程式碼的所有可能執行路徑

Denial-of-Service (DoS)

- **定義:** 拒絕服務攻擊相關的安全問題
- **風險:** 可能導致應用程式無法正常服務或系統資源耗盡
- **常見情況:**
 - 無限迴圈
 - 記憶體洩漏
 - 除零錯誤 (如您的案例)
 - 過度資源消耗
- **意義:** 此問題可能被惡意利用來癱瘓系統

CERT (Computer Emergency Response Team)

- **定義:** 電腦緊急應變小組制定的安全編碼標準
- **目的:** 提供安全程式設計的最佳實踐和指導原則

- 涵蓋範圍:
 - Java 安全編碼標準
 - C/C++ 安全編碼標準
 - 其他程式語言的安全實踐
- 意義: 違反了 CERT 制定的安全編碼規範

Vulnerability (安全漏洞)

- 定義: 可能被惡意攻擊者利用的安全弱點
- 影響: 安全風險，可能導致資料洩漏、系統被入侵
- 範例:

```
// SQL Injection 風險
String query = "SELECT * FROM users WHERE id = " + userId;

// 硬編碼密碼
String password = "admin123";

// 不安全的隨機數生成
Random random = new Random();
```

Code Smell (程式碼整潔)

- 定義: 程式碼品質問題，不會影響功能但會影響維護性
- 影響: 可讀性差、難以維護、技術債務累積
- 範例:

```
// 未使用的 import
import org.springframework.security.core.userdetails.UserDetails;

// 過長的方法
public void processData() {
    // 100+ 行程式碼
}

// 重複的程式碼塊
if (user.getAge() > 18) {
    // 相同邏輯在多處重複
}
```

▲ Severity 嚴重程度說明

🚫 Blocker (阻斷級)

- 影響: 必須立即修復，會阻止應用程式正常運作
- 建議: 不允許發布到生產環境
- 範例: 嚴重的空指標異常、死鎖風險

🔴 Critical (關鍵級)

- 影響: 非常嚴重，可能導致應用程式在生產環境出現問題
- 建議: 發布前必須修復
- 範例: 記憶體洩漏、SQL 注入漏洞

🟡 Major (主要級)

- 影響: 重要問題，會影響開發效率或存在潛在風險
- 建議: 應該優先修復
- 範例: 未使用的變數、複雜度過高的方法

🟡 Minor (次要級)

- 影響: 輕微問題，主要影響程式碼品質
- 建議: 可以排程修復
- 範例: 命名規範不符、註釋不足

📘 Info (資訊級)

- 影響: 提醒事項，不影響功能但值得注意
- 建議: 參考改善
- 範例: 程式碼風格建議、最佳實踐提醒

Quality Gate 建議設定

```
# 建議的品質閘門標準
quality_gate:
  coverage: >= 80%           # 測試覆蓋率
  duplicated_lines: < 3%      # 重複程式碼率
  blocker_violations: 0       # 阻斷級問題必須為 0
  critical_violations: 0      # 關鍵級問題必須為 0
  major_violations: < 10     # 主要級問題少於 10 個
  security_rating: A          # 安全評級必須是 A
  maintainability_rating: A  # 可維護性評級必須是 A
```

建議先從 Critical 和 Major 級別的問題開始修復